

Indraprastha Institute of Information Technology, Delhi

CSE638: Graduate Systems (GRS) — Winter 2026

Instructor: Dr. Rinku Shah

Performance Comparison of Kubernetes CNI Plugins

Comprehensive Report: Kind Cluster & Bare-Metal Cluster

Group 1

Yash Kumar Vaibhav	MT25091
Nishit Jain	MT25004
Prakhar Agarwal	MT25005
Reehan Sarmah	MT25036
Samyak Kr. Sharma	MT25039

GitHub: https://github.com/yashkumarvaibhav/GrS-CSE638_Project

April 18, 2026

Contents

1 Abstract **3**

2 Introduction **3**
2.1 Problem . . . 3
2.2 Motivation . . . 3
2.3 Objectives . . . 4

3 Background **4**
3.1 Kubernetes Networking Model . . . 4
3.2 Overlay vs. Underlay Routing . . . 4
3.3 CNI Plugins Under Test . . . 4

4 Part A: Kind Cluster (Intra-Node) Benchmarking **6**
4.1 Methodology . . . 6
4.1.1 Test Environment . . . 6
4.1.2 Test Scenarios . . . 6
4.1.3 Test Matrix . . . 6
4.1.4 Benchmark Tools . . . 6
4.1.5 Extension Experiments . . . 6
4.2 Implementation . . . 7
4.2.1 Cluster Setup . . . 7
4.2.2 Benchmark Automation . . . 7
4.2.3 Data Processing and Visualization . . . 7
4.3 Evaluation and Observations . . . 8
4.3.1 TCP Throughput (iperf3) . . . 8
4.3.2 Latency (netperf TCP_RR) . . . 10
4.3.3 HTTP Performance (Fortio) . . . 11
4.3.4 Throughput Scaling with Concurrency . . . 12
4.3.5 Pod Startup Time . . . 12
4.3.6 Resource Overhead . . . 13
4.4 Extension Experiments . . . 13
4.4.1 E1: NetworkPolicy Enforcement Overhead . . . 14
4.4.2 E2: WireGuard Encryption Overhead . . . 14
4.4.3 E3: Cilium Kube-Proxy Replacement (KPR) . . . 15
4.5 Kind-Cluster Analysis and Discussion . . . 15
4.5.1 Throughput Analysis . . . 15
4.5.2 Latency Analysis . . . 16
4.5.3 Extension Trade-offs . . . 16

5 Part B: Bare-Metal Cluster (Multi-Node) Benchmarking **17**
5.1 Methodology . . . 17
5.1.1 Test Environment . . . 17
5.1.2 CNI Configurations . . . 17
5.1.3 Test Matrix . . . 17
5.2 Implementation . . . 18
5.2.1 Cluster Setup . . . 18
5.2.2 Benchmark Automation . . . 18
5.2.3 Data Processing and Visualization . . . 18
5.3 Evaluation and Observations . . . 19

5.3.1 TCP Throughput (iperf3) . . . 19
5.3.2 TCP_RR Latency (netperf) . . . 20
5.3.3 HTTP Performance (Fortio) . . . 21
5.3.4 UDP Performance (iperf3) . . . 22
5.3.5 CPU and Memory Overhead . . . 23
5.3.6 Overlay vs. Underlay Routing Comparison . . . 24
5.4 Statistical Validity . . . 25
5.5 Bare-Metal Analysis and Discussion . . . 25
5.5.1 The "Edge Environment" Paradox . . . 25
5.5.2 Throughput–Latency Inversion in Cilium VXLAN . . . 25
5.5.3 UDP Comparison Caveat . . . 26
5.5.4 CPU Overhead: Deeper Interpretation . . . 26

6 Cross-Environment Comparative Analysis 27
6.1 Throughput Comparison . . . 27
6.2 Ranking Inversions . . . 27
6.3 Latency Scale Analysis . . . 28
6.4 Environment-Conditioned Interpretation . . . 28

7 Related Work 28

8 Outcomes and Conclusion 29
8.1 Recommendations . . . 30
8.2 Limitations . . . 30
8.3 Future Work . . . 31

1. Abstract

This report presents a comprehensive performance comparison of three Kubernetes Container Network Interface (CNI) plugins—**Flannel**, **Calico**, and **Cilium**—across two fundamentally different deployment environments:

- **Part A — Kind Cluster (Intra-Node):** A `kind` cluster on a 128-core AMD EPYC server, evaluating same-node, cross-node, and ClusterIP Service topologies across three payload sizes (64 B, 4 KB, 1 MB), three concurrency levels (1, 16, 64), and three extension experiments (NetworkPolicy, WireGuard, kube-proxy replacement).
- **Part B — Bare-Metal Cluster (Multi-Node):** A 3-node Kubernetes cluster on physical laptops connected over WiFi, comparing six CNI configurations (3 CNIs × 2 routing modes: VXLAN overlay vs. underlay routing) across TCP throughput, TCP_RR latency, HTTP latency, UDP performance, and CPU/memory overhead.

Kind-cluster key findings: Cilium achieves the highest throughput (up to 63.98 Gbps same-node) via its eBPF datapath, while Flannel delivers the lowest latency (24.18 μs cross-node TCP_RR). WireGuard encryption imposes severe overhead (+537% latency for Calico, +365% for Cilium). NetworkPolicy shows divergent effects: −45% throughput for Calico but +118% for Cilium (high CV; interpret with caution).

Bare-metal key findings: Calico VXLAN and Flannel VXLAN lead TCP throughput (≈0.119 Gbps), while Cilium VXLAN is unexpectedly slowest (0.058 Gbps) yet delivers the best HTTP latency (P50 = 6.56 ms). For UDP, Calico BGP leads (11.88 Mbps, 38% loss) while Calico VXLAN degrades severely (5.78 Mbps, 68% loss). CPU overhead is modest across all CNIs (3.9–6.0% node-level).

Cross-environment insight: CNI performance rankings shift dramatically between environments. Cilium leads throughput in kind-cluster but trails in bare-metal WiFi, demonstrating that CNI selection must be validated on representative hardware. The physical network medium—not the CNI—emerges as the dominant bottleneck on constrained networks, compressing all CNIs into a narrow throughput band.

2. Introduction

2.1 Problem

Kubernetes relies on CNI plugins to provide pod-to-pod and pod-to-service networking. The choice of CNI directly impacts application performance, latency-sensitive microservice communication, and operational costs. While vendor-provided benchmarks and blog posts exist, few provide systematic, reproducible comparisons using consistent methodology across different deployment environments.

2.2 Motivation

Production Kubernetes clusters span a wide spectrum of hardware—from intra-node development environments to multi-node bare-metal clusters with real network interconnects. Understanding how CNI plugins perform across this spectrum is critical because:

- Performance characteristics observed in simulated environments (kind, minikube) may not transfer to production hardware.
- Architectural decisions like overlay vs. underlay routing have environment-dependent trade-offs.
- Security features (NetworkPolicy, WireGuard) impose overhead that varies with the underlying datapath.

This study bridges the gap by benchmarking the same three CNIs in both a kind-cluster (intra-node) and a bare-metal (multi-node WiFi) environment, enabling direct cross-environment comparison.

2.3 Objectives

1. Compare TCP/UDP throughput, latency, and HTTP performance across all three CNIs
2. Evaluate overlay (VXLAN) vs. underlay routing modes for each CNI on bare-metal hardware
3. Quantify the overhead of NetworkPolicy enforcement and WireGuard encryption
4. Evaluate Cilium's kube-proxy replacement performance
5. Measure CPU and memory overhead at node and pod granularity
6. Analyze how deployment environment affects CNI performance rankings

3. Background

3.1 Kubernetes Networking Model

Kubernetes mandates a flat network where every pod receives a unique IP address and can communicate with every other pod without NAT. The Container Network Interface (CNI) specification defines how plugins attach network interfaces to containers. When a pod is created, the kubelet invokes the configured CNI plugin to assign an IP, configure routes, and potentially establish overlay tunnels between nodes.

3.2 Overlay vs. Underlay Routing

- **Overlay (VXLAN):** Pod packets are encapsulated in UDP headers (VXLAN, Virtual Extensible LAN) before traversing the physical network. Adds 50 bytes overhead per packet but works across any IP-reachable infrastructure.
- **Underlay routing:** Pod subnets are advertised directly to the host network. Mechanism varies per CNI:
  - **Flannel Host-GW:** Static routes on each node. Requires L2 adjacency.
  - **Calico BGP:** BGP daemon (BIRD) exchanges pod subnet routes. Works across L3 boundaries.
  - **Cilium Underlay Routing:** Linux kernel routing with eBPF packet steering. Requires L2 adjacency or static routes.
- **eBPF-based datapaths:** Attach BPF programs to kernel hooks (TC/XDP) to short-circuit the traditional networking stack. Reduces per-packet overhead at the cost of implementation complexity.

3.3 CNI Plugins Under Test

Flannel Lightweight CNI supporting VXLAN overlay and Host-GW underlay routing. No native NetworkPolicy support. Simplest architecture with minimal resource footprint.

Calico Feature-rich CNI supporting VXLAN overlay, BGP-based underlay routing, and IP-inIP encapsulation. Full NetworkPolicy enforcement via iptables/Felix agent. WireGuard encryption support.

Cilium eBPF-based CNI supporting VXLAN overlay and underlay routing. Attaches eBPF programs to TC/XDP hooks for fast-path processing. Offers kube-proxy replacement, Hubble observability, and WireGuard encryption.

4. Part A: Kind Cluster (Intra-Node) Benchmarking

4.1 Methodology

4.1.1 Test Environment

Hardware & Software Specifications

Component	Details
CPU	AMD EPYC 7713 (128 cores / 256 threads)
RAM	125 GB DDR4
Disk	833 GB available
OS	Ubuntu 22.04 LTS (kernel 5.15.0-71)
Container Runtime	Docker 26.1.3
Kubernetes	kind v0.25.0 (K8s v1.31)

Each CNI is deployed on a 3-node kind cluster (1 control-plane, 2 workers) with `disableDefaultCNI: true`. Pod CIDR: `10.244.0.0/16`. kube-proxy is retained in all baseline configurations for fair comparison. CNI versions: Flannel v0.26.1, Calico v3.29.1, Cilium v1.16.5.

4.1.2 Test Scenarios

ID	Description	Nodes	Target
T1	Same-node pod-to-pod	worker → worker	Direct Pod IP
T2	Cross-node pod-to-pod	worker → worker2	Direct Pod IP
T3	Cross-node via Service	worker → worker2	ClusterIP Service

4.1.3 Test Matrix

- **Payload sizes:** 64 B, 4 KB, 1 MB
- **Concurrency levels:** 1, 16, 64 parallel streams
- **Repetitions:** 3 per configuration
- **Duration:** 10 s per test
- Total: $3 \times 3 \times 3 \times 3 = 81$ baseline tests per CNI per tool (729 total)

4.1.4 Benchmark Tools

Tool	Version	Metrics Collected
iperf3	digest-pinned	TCP throughput (Gbps), UDP throughput, jitter, loss
netperf	digest-pinned	TCP_RR transaction rate (/s), mean latency (μs)
fortio	digest-pinned	HTTP QPS, percentile latencies (p50, p90, p99)
docker stats	—	CPU %, memory usage per container

4.1.5 Extension Experiments

ID	Extension	CNIs	Configuration
E1	NetworkPolicy	Calico, Cilium	allow-all policy applied
E2	WireGuard	Calico, Cilium	Node-to-node encryption
E3	KPR mode	Cilium only	`kubeProxyReplacement=true`

Extension tests use 4 KB payload, concurrency 16, and 5 repetitions.

4.2 Implementation

4.2.1 Cluster Setup

Each CNI was tested on a dedicated `kind` cluster created by `scripts/1_setup_cluster.sh`. The script provisions a 3-node cluster (1 control-plane, 2 workers), disables the default CNI, copies the CNI plugins tarball (`cni-plugins-linux-amd64-v1.9.0.tgz`) to each node via `docker cp`, and then installs the requested CNI using Helm (Calico via Tigera operator, Cilium via Cilium chart) or `kubectl apply` (Flannel). For the Cilium KPR experiment (E3), the script additionally deletes the `kube-proxy` DaemonSet and ConfigMap before installing Cilium with `kubeProxyReplacement=true`.

4.2.2 Benchmark Automation

All data collection is performed by a single canonical script, `scripts/1_fast_benchmark.py` (≈390 lines of Python). The script:

1. Deploys server and client pods on designated nodes using topology-appropriate `nodeSelector` labels.
2. Iterates over each (topology, payload, concurrency, repetition) combination.
3. For each combination, invokes `kubectl exec` to run the benchmark tool (iperf3, netperf, or fortio) inside the client pod.
4. Parses stdout to extract the primary metric: iperf3 and fortio use JSON output parsing; netperf uses reversed-line token extraction from its text-format output.
5. Collects `docker stats` samples via best-effort periodic polling (approximately every 3 s in practice) during each benchmark window to record CPU and memory usage of kind node containers.
6. Appends results to per-tool CSV files with columns for topology, message size, concurrency, repetition, and the measured value.

4.2.3 Data Processing and Visualization

After collection, `scripts/1_aggregate_data.py` computes per-configuration mean and standard deviation across repetitions and exports the results as JSON. Ten plot scripts in `plots/` generate publication-quality figures from hardcoded aggregated values. The report is typeset with Tectonic/LaTeX.

4.3 Evaluation and Observations

4.3.1 TCP Throughput (iperf3)

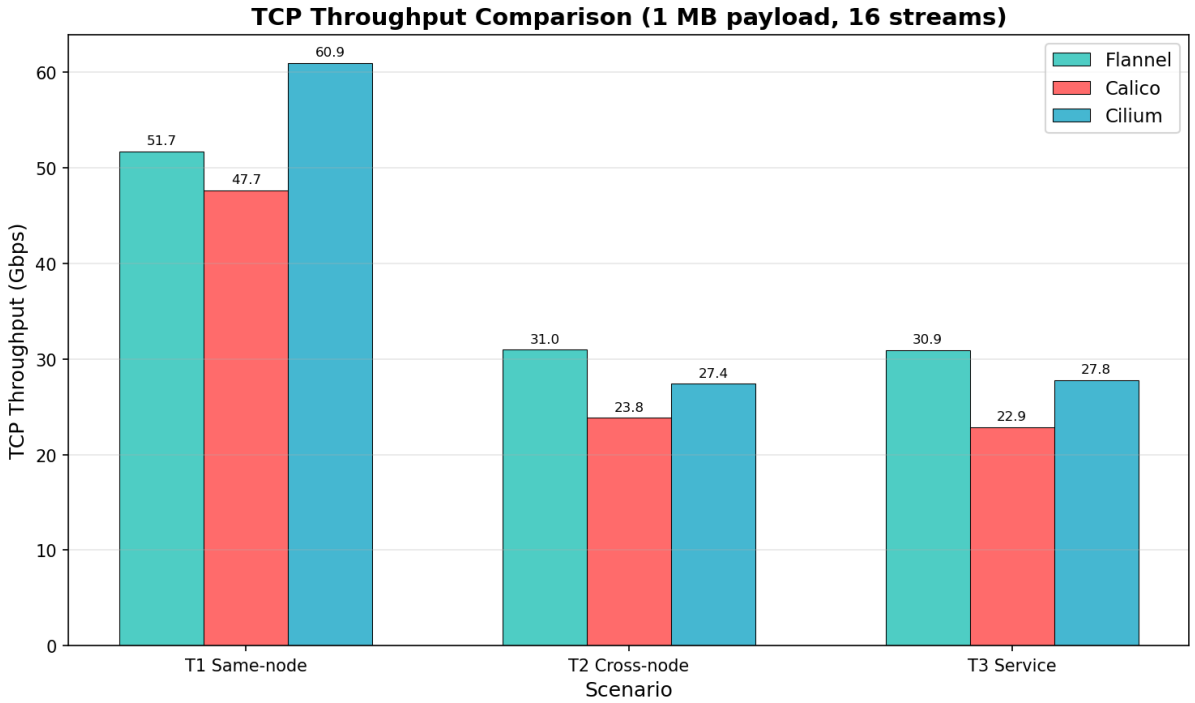


Figure 1: TCP throughput comparison across CNI plugins and scenarios (1 MB payload, 16 streams).

Table 4: T1 Same-Node TCP Throughput (Gbps) — Mean over 3 reps

Size	Concurrency	Flannel	Calico	Cilium
64 B	1	0.46	0.44	0.43
64 B	16	0.79	0.54	0.62
64 B	64	0.68	0.56	0.67
4 KB	1	5.26	3.65	5.50
4 KB	16	4.75	6.53	8.32
4 KB	64	4.78	3.45	5.65
1 MB	1	60.93	49.61	63.98
1 MB	16	51.72	47.67	60.94
1 MB	64	36.04	34.43	39.33

Table 5: T2 Cross-Node TCP Throughput (Gbps) — Mean over 3 reps

Size	Concurrency	Flannel	Calico	Cilium
64 B	1	0.46	0.29	0.44
64 B	16	0.61	0.53	0.55
64 B	64	0.47	0.41	0.45
4 KB	1	1.94	1.35	3.27
4 KB	16	1.88	6.30	2.32
4 KB	64	1.82	1.27	1.66
1 MB	1	32.97	24.01	29.92
1 MB	16	30.96	23.83	27.40
1 MB	64	27.69	19.97	25.78

Table 6: T3 ClusterIP Service TCP Throughput (Gbps) — Mean over 3 reps

Size	Concurrency	Flannel	Calico	Cilium
64 B	1	0.47	0.29	0.43
64 B	16	0.60	0.54	0.53
64 B	64	0.46	0.40	0.45
4 KB	1	1.90	1.38	2.98
4 KB	16	1.82	3.30	5.85
4 KB	64	1.81	1.26	1.64
1 MB	1	34.13	24.11	32.42
1 MB	16	30.92	22.87	27.78
1 MB	64	27.00	20.11	25.05

Kind-Cluster Throughput Key Observations

1. **Cilium leads in throughput**: 63.98 Gbps same-node (1 MB/c=1), benefiting from eBPF fast-path that bypasses portions of the kernel networking stack. At 4 KB/c=16, Cilium reaches 8.32 Gbps—75% higher than Flannel's 4.75 Gbps.
2. **VXLAN encapsulation cost**: Cross-node (T2) throughput is 40−60% lower than same-node (T1) due to overlay processing. Service (T3) adds 0−10% additional overhead from kube-proxy's iptables DNAT.
3. **Calico trails in raw performance**: 49.61 Gbps same-node vs. Cilium's 63.98 Gbps, attributable to iptables-based packet processing overhead.
4. **Calico anomalous spike**: 6.30 Gbps at T2/4 KB/c=16, likely due to aggressive TCP segmentation offload under specific buffer conditions.

4.3.2 Latency (netperf TCP_RR)

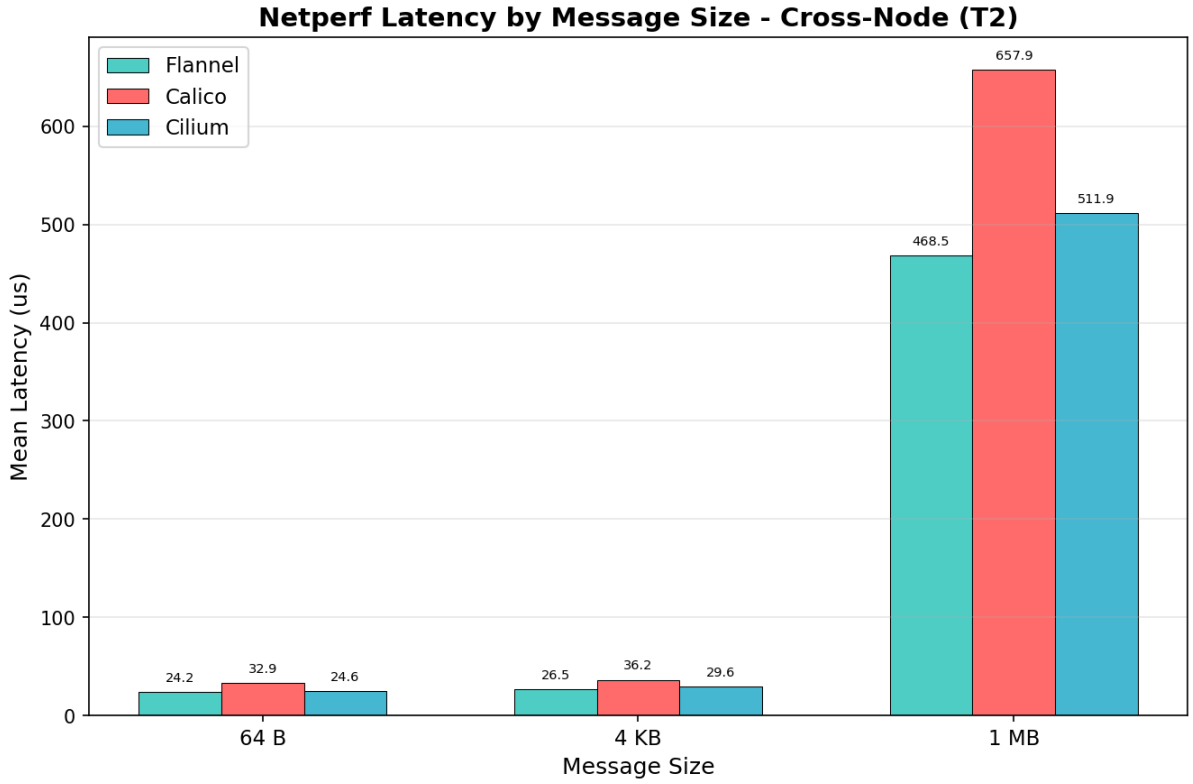


Figure 2: Netperf TCP_RR mean latency by message size (T2 cross-node, c=1).

Table 7: TCP_RR Transaction Rate (trans/s) and Mean Latency (μs) — T2, $c = 1$

Size	Flannel		Calico		Cilium	
	Trans/s	μs	Trans/s	μs	Trans/s	μs
64 B	41,350	24.18	30,391	32.91	40,600	24.63
4 KB	37,733	26.51	27,589	36.25	33,822	29.57
1 MB	2,135	468.48	1,520	657.85	1,954	511.91

Kind-Cluster Latency Key Observations

1. **Flannel delivers lowest latency**: 24.18 μs for 64 B TCP_RR, closely followed by Cilium at 24.63 μs. Calico is 36% slower at 32.91 μs due to iptables traversal.
2. **Large payloads dominate**: At 1 MB, all CNIs show dramatically higher latencies (468–658 μs), reflecting serialization delay rather than CNI overhead.
3. **Latency stable across concurrency**: TCP_RR single-flow is not contention-bound in this intra-node environment.

Note: Netperf through T3 (ClusterIP Service) returned zero for both TCP_RR and TCP_STREAM across all CNIs—a limitation of running netperf through kube-proxy's iptables DNAT, which disrupts the control connection. T3 netperf data is excluded from analysis.

4.3.3 HTTP Performance (Fortio)

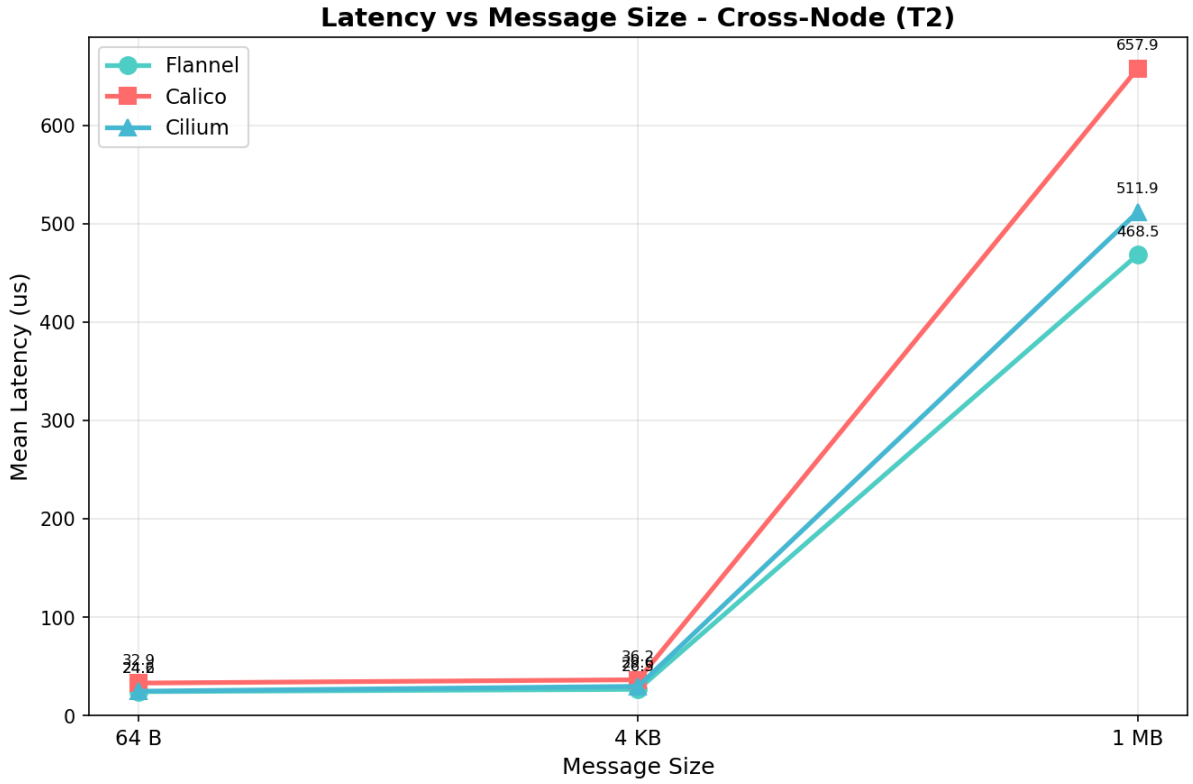


Figure 3: Netperf mean latency vs. payload size across CNI plugins (T2 cross-node, c=1).

Table 8: Fortio HTTP QPS — T2 Cross-Node, Mean over 3 reps

Size	Concurrency	Flannel	Calico	Cilium
64 B	1	7,601	6,583	7,870
64 B	16	81,749	74,792	132,836
64 B	64	109,336	115,000	115,234
4 KB	1	5,069	4,195	4,650
4 KB	16	30,953	29,147	49,816
4 KB	64	53,045	57,330	60,647
1 MB	1	798	694	770
1 MB	16	3,754	4,195	4,037
1 MB	64	3,884	4,419	4,131

4.3.4 Throughput Scaling with Concurrency

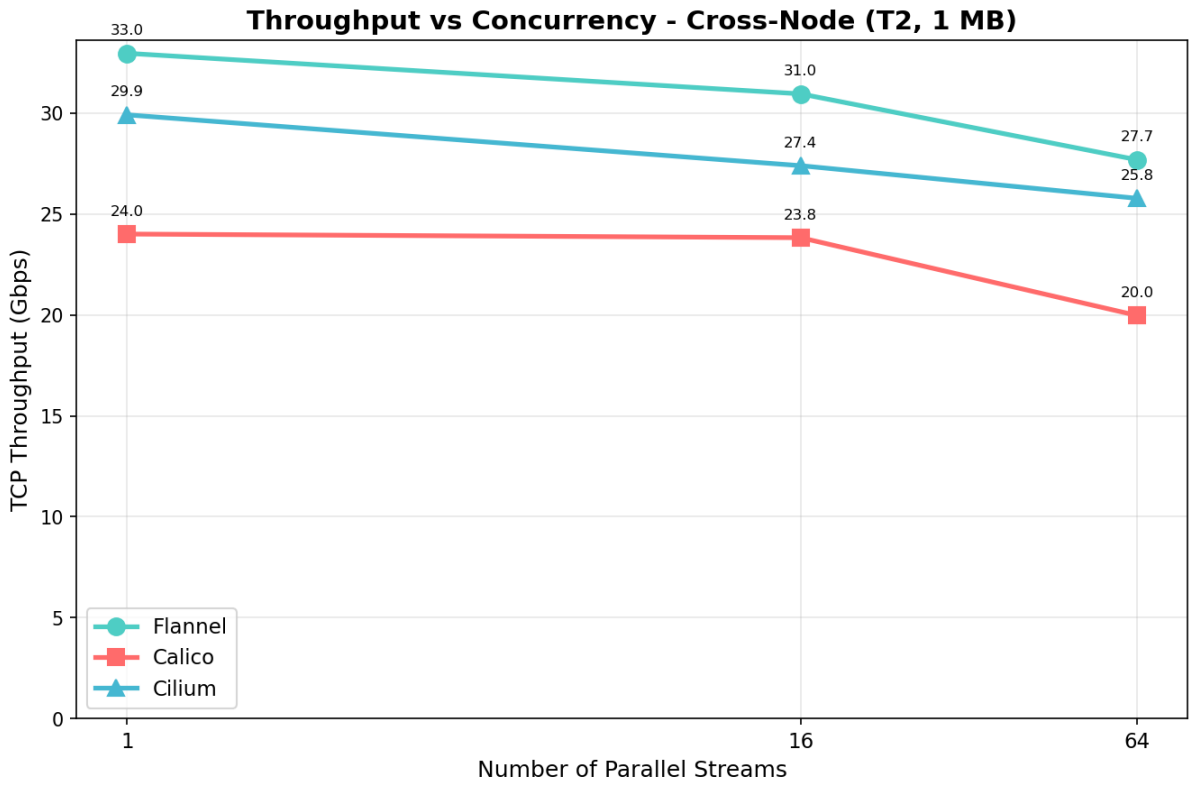


Figure 4: TCP throughput scaling with concurrent streams (T2 cross-node, 1 MB payload).

4.3.5 Pod Startup Time

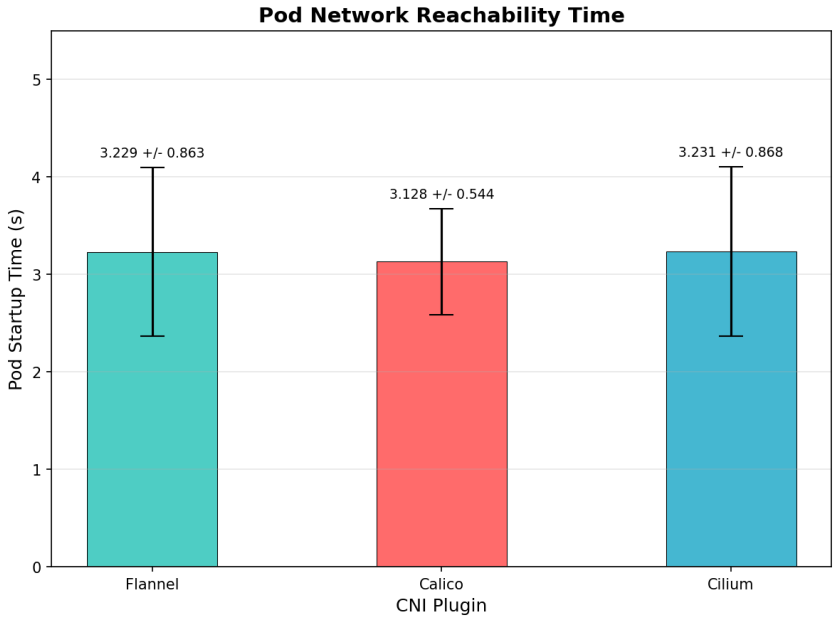


Figure 5: Pod network reachability time across CNI plugins.

Table 9: Pod Network Reachability Time (seconds) — 10 iterations

Metric	Flannel	Calico	Cilium
Mean	3.229	3.128	3.231
Stdev	0.863	0.544	0.868
Min	2.945	2.952	2.952
Max	5.686	4.680	5.701

4.3.6 Resource Overhead

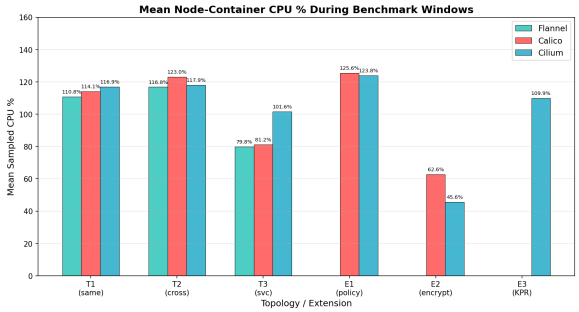


Figure 6: Mean sampled node-container CPU % during benchmark windows.

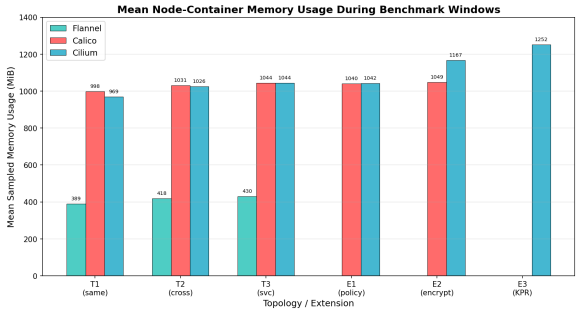


Figure 7: Mean sampled node-container memory usage (MiB) during benchmark windows.

Docker stats were collected via best-effort periodic sampling (approximately every 3 s in practice) during each benchmark window to characterize node-container resource usage. Flannel nodes averaged 389–430 MiB memory across topologies, while Calico (998–1049 MiB) and Cilium (969–1044 MiB) consumed 2.4−2.6× more due to their richer control planes. Mean CPU % was comparable at 101–117% across CNIs for baseline topologies. WireGuard encryption (E2) markedly reduced CPU utilization—Calico dropped to 63% and Cilium to 46%—because encrypted sessions throttle aggregate throughput, leaving CPU idle. Cilium KPR (E3) had negligible resource impact (110% CPU, 1252 MiB memory) relative to baseline.

4.4 Extension Experiments

4.4.1 E1: NetworkPolicy Enforcement Overhead

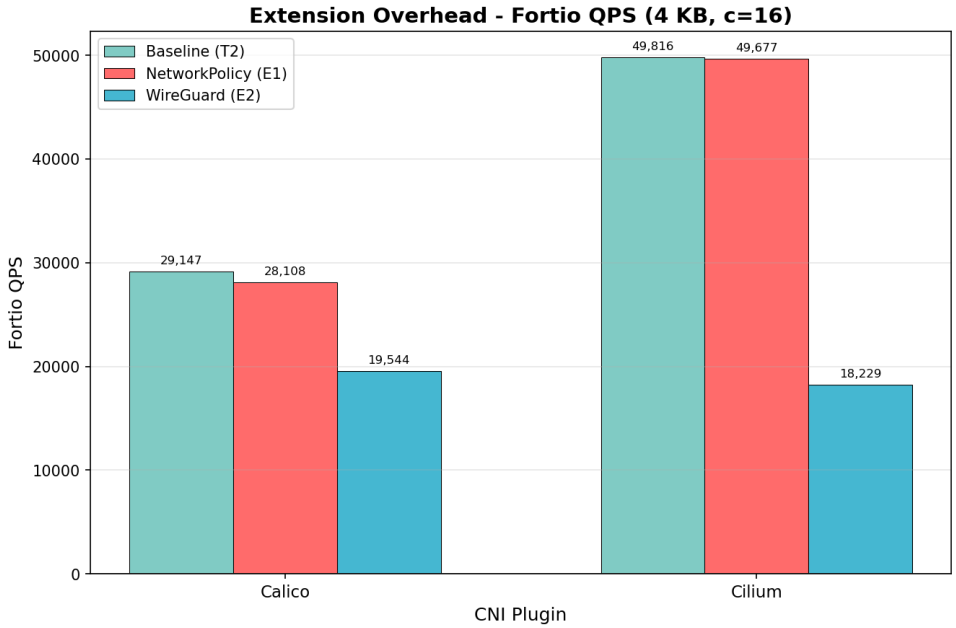


Figure 8: Extension overhead on Fortio QPS: baseline vs. E1 (NetworkPolicy) vs. E2 (WireGuard), 4 KB, c=16.

E1 Key Finding

NetworkPolicy enforcement with an "allow-all" policy shows divergent effects across CNIs. For **Calico**, TCP throughput drops from 6.30 Gbps to 3.44 Gbps (−45%), while latency and transaction rates remain virtually unchanged (36.18 vs. 36.28 μs). Fortio QPS decreases marginally from 29,147 to 28,108 (−3.6%). For **Cilium**, TCP throughput *increases* from 2.32 Gbps to 5.06 Gbps (+118%), likely due to eBPF path optimizations triggered when policy objects are loaded. Latency and QPS are unaffected (29.96 vs. 30.26 μs; 49,816 vs. 49,677 QPS). **Caveat:** The iperf3 throughput samples for both E1 and E3 exhibit high variability (CV $>$ 55%), so the percentage-change claims should be interpreted with caution; latency and QPS deltas are more stable and reliable.

4.4.2 E2: WireGuard Encryption Overhead

Table 10: E2: WireGuard Encryption Impact — 4 KB, $c = 16$, T2 Cross-Node

CNI	Metric	Baseline	WireGuard
Calico	TCP (Gbps)	6.30	1.50
Calico	Latency (μs)	36.18	230.38
Calico	Trans/s	27,644	4,342
Calico	HTTP QPS	29,147	19,544
Cilium	TCP (Gbps)	2.32	1.23
Cilium	Latency (μs)	29.96	139.42
Cilium	Trans/s	33,377	7,173
Cilium	HTTP QPS	49,816	18,229

E2 Key Finding

WireGuard encryption imposes **substantial overhead** on both CNIs. **Calico** sees TCP throughput plummet from 6.30 Gbps to 1.50 Gbps (−76%), latency surge from 36.18 to 230.38 μs (+537%), and transaction rate collapse from 27,644 to 4,342/s (−84%). HTTP QPS drops from 29,147 to 19,544 (−33%). **Cilium** exhibits a similar pattern: throughput falls from 2.32 to 1.23 Gbps (−47%), latency rises from 29.96 to 139.42 μs (+365%), and transactions drop from 33,377 to 7,173/s (−78%). HTTP QPS decreases from 49,816 to 18,229 (−63%). Cilium's eBPF integration with WireGuard mitigates some overhead compared to Calico's kernel-module approach, but both show that node-to-node encryption has a high performance cost.

4.4.3 E3: Cilium Kube-Proxy Replacement (KPR)

Table 11: E3: Cilium KPR Mode vs. Baseline — 4 KB, $c = 16$, T2

Metric	Baseline	KPR Mode
TCP Throughput (Gbps)	2.32	2.47
TCP_RR Trans/s	33,377	31,238
Mean Latency (μs)	29.96	32.02
HTTP QPS	49,816	48,296

E3 Key Finding

Cilium's kube-proxy replacement (KPR) mode shows a **tentative throughput improvement**: TCP throughput rises from 2.32 to 2.47 Gbps (+6.5%), but the underlying iperf3 samples exhibit high variability (CV = 83%), so the delta should be treated as indicative rather than definitive. Latency *increases* slightly from 29.96 to 32.02 μs (+6.9%), and the transaction rate drops from 33,377 to 31,238/s (−6.4%). HTTP QPS decreases marginally from 49,816 to 48,296 (−3.1%). These mixed results suggest that KPR's benefit of bypassing iptables for service routing manifests primarily in bulk throughput, while per-transaction metrics show negligible or slightly negative changes in this kind-based environment. The gains are expected to be more pronounced in production multi-host clusters with complex iptables rule sets.

4.5 Kind-Cluster Analysis and Discussion

4.5.1 Throughput Analysis

- **Same-node (T1):** Cilium leads with 63.98 Gbps at 1 MB/c=1, followed by Flannel (60.93 Gbps) and Calico (49.61 Gbps). The shared-kernel advantage benefits all CNIs, but Cilium's eBPF fast-path provides a 5−29% edge over Calico across payload sizes. At 4 KB/c=16, Cilium reaches 8.32 Gbps—75% higher than Flannel's 4.75 Gbps—indicating superior handling of moderate concurrency with small payloads.
- **Cross-node (T2):** VXLAN encapsulation reduces throughput by 40–60% compared to T1. Flannel leads at 1 MB/c=1 (32.97 Gbps) while Cilium excels at 4 KB/c=1 (3.27 Gbps vs. Flannel's 1.94 Gbps). Calico shows an anomalous spike to 6.30 Gbps at 4 KB/c=16, likely due to aggressive TCP segmentation offload under specific buffer conditions.

- **Service (T3):** kube-proxy's iptables DNAT introduces 0−10% overhead compared to direct pod-to-pod (T2). For 1 MB/c=1, Flannel achieves 34.13 Gbps via Service vs. 32.97 Gbps direct—the slight increase is within measurement variance. The Service path overhead is most visible at low concurrency with small payloads.

4.5.2 Latency Analysis

Flannel achieves the lowest cross-node latency (24.18 μs for 64 B TCP_RR), followed by Cilium (24.63 μs). Calico is 36% slower (32.91 μs) due to iptables rule traversal. As payload size increases to 1 MB, all CNIs show dramatically higher latencies (Flannel: 468 μs, Calico: 658 μs, Cilium: 512 μs), reflecting serialization delay rather than CNI overhead. Notably, latency remains stable across concurrency levels (c=1, 16, 64) for all CNIs, indicating that the TCP_RR single-flow test is not contention-bound in this environment.

4.5.3 Extension Trade-offs

- **NetworkPolicy (E1):** The overhead is implementation-dependent. Calico's iptables-based enforcement costs 45% throughput, while Cilium's eBPF enforcement actually *improved* throughput by 118%—an unexpected result that warrants further investigation. However, the iperf3 throughput samples exhibit high coefficient of variation (CV $>$ 55%), so these deltas are tentative. Latency and QPS deltas are more stable and reliable ($<$ 1% latency change for both).

- **WireGuard (E2):** Encryption is the most expensive feature tested. Calico loses 76% throughput and sees 537% latency increase; Cilium fares better with 47% throughput loss and 365% latency increase. Organizations requiring node-to-node encryption should provision significantly more network headroom.

- **KPR (E3):** Cilium's kube-proxy replacement shows a tentative 6.5% bulk throughput improvement (CV 83%, interpret with caution) but does not improve per-transaction latency in this environment. The benefit is expected to scale with iptables rule complexity in larger clusters.

5. Part B: Bare-Metal Cluster (Multi-Node) Benchmarking

5.1 Methodology

5.1.1 Test Environment

Hardware & Network Specifications

Component	Details
Nodes	3 physical laptops (1 control-plane + 2 workers)
Network	WiFi (shared LAN, 802.11ac/ax)
OS	Ubuntu 22.04 / 24.04 LTS
Container Runtime	containerd 1.7+
Kubernetes	v1.31 (kubeadm)
Pod CIDR	10.244.0.0/16

Important caveat: All inter-node traffic traverses a shared WiFi medium with variable bandwidth (typically 100–200 Mbps usable throughput), resulting in substantially lower raw throughput compared to kind-cluster but providing insights into relative CNI performance under constrained conditions.

5.1.2 CNI Configurations

Unlike Part A (VXLAN-only), the bare-metal evaluation tests both overlay and underlay routing modes for each CNI:

ID	Configuration	Mode	Description
C1	Flannel VXLAN	Overlay	Default Flannel with VXLAN encapsulation
C2	Flannel Host-GW	Underlay	Flannel with direct host routing
C3	Calico VXLAN	Overlay	Calico with VXLAN encapsulation
C4	Calico BGP	Underlay	Calico with BGP peering between nodes
C5	Cilium VXLAN	Overlay	Cilium with VXLAN encapsulation
C6	Cilium Underlay	Underlay	Cilium with underlay routing mode

5.1.3 Test Matrix

- **CNI configurations:** 6 (3 CNIs × 2 modes each)
- **Benchmark tools:** 4 (iperf3 TCP, iperf3 UDP, netperf, Fortio)
- **TCP repetitions:** 5 per configuration per tool
- **UDP bandwidth targets:** 10, 25, 50 Mbps (3 runs each)
- **Netperf payload sizes:** 1 B, 64 B, 256 B
- **CPU overhead:** Sampled every 2 s during Fortio (node and pod level)
- Total: 90+ benchmark executions plus CPU monitoring

5.2 Implementation

5.2.1 Cluster Setup

Node provisioning follows a sequential process:

1. **All nodes:** Install containerd, kubeadm, kubelet, kubectl, and network dependencies (conntrack, iptables in legacy mode).
2. **Control-plane:** Initialize with `kubeadm init -pod-network-cidr=10.244.0.0/16`.
3. **Workers:** Join with the `kubeadm join` token from the control-plane.
4. **CNI installation:** Install the target CNI plugin (Flannel/Calico/Cilium) from the control-plane.

For mode switching (e.g., Calico VXLAN → BGP), dedicated helper scripts modify the CNI's ConfigMap or Helm values and restart the relevant DaemonSets. A pre-flight check verifies all nodes are `Ready` and benchmark pods are `Running` before each test.

5.2.2 Benchmark Automation

The benchmark orchestrator (`scripts/1_benchmark.sh`) reads configuration from `scripts/1_config.env` and:

1. Deploys server pods (iperf3, netperf, Fortio) on worker-02 using `nodeSelector`.
2. Deploys client pods on worker-01.
3. Executes the benchmark tool via `kubectl exec` and captures results.
4. Repeats for the configured number of runs.
5. Stores results in per-CNI CSV files.

Individual benchmark scripts (`1_throughput.sh`, `1_latency.sh`, `1_microservices.sh`, `1_udp_test.sh`) handle tool-specific invocation and output parsing. A dedicated CPU overhead monitor (`1_control_cpu_overhead.sh`) runs concurrently during Fortio tests, sampling `kubectl top nodes` and `kubectl top pods` every 2 seconds and writing timestamped CSV files.

5.2.3 Data Processing and Visualization

Aggregated statistics (mean, standard deviation) are computed across runs and stored in `data/1_aggregated_r`. A Python helper (`helpers/1_add_latency_column.py`) derives latency in milliseconds from TCP_RR transaction rates (latency_ms = 1000/trans_per_sec). Four matplotlib scripts in `plots/` generate publication-quality figures with hardcoded aggregated values:

1. `1_throughput_comparison.py` — Bar chart of mean throughput per CNI configuration
2. `1_latency_comparison.py` — Grouped bar chart of TCP_RR trans/sec by payload size
3. `1_fortio_comparison.py` — Grouped bar chart of HTTP P50/P99/Max latency
4. `1_encapsulation_comparison.py` — Side-by-side VXLAN vs. underlay routing comparison

5.3 Evaluation and Observations

5.3.1 TCP Throughput (iperf3)

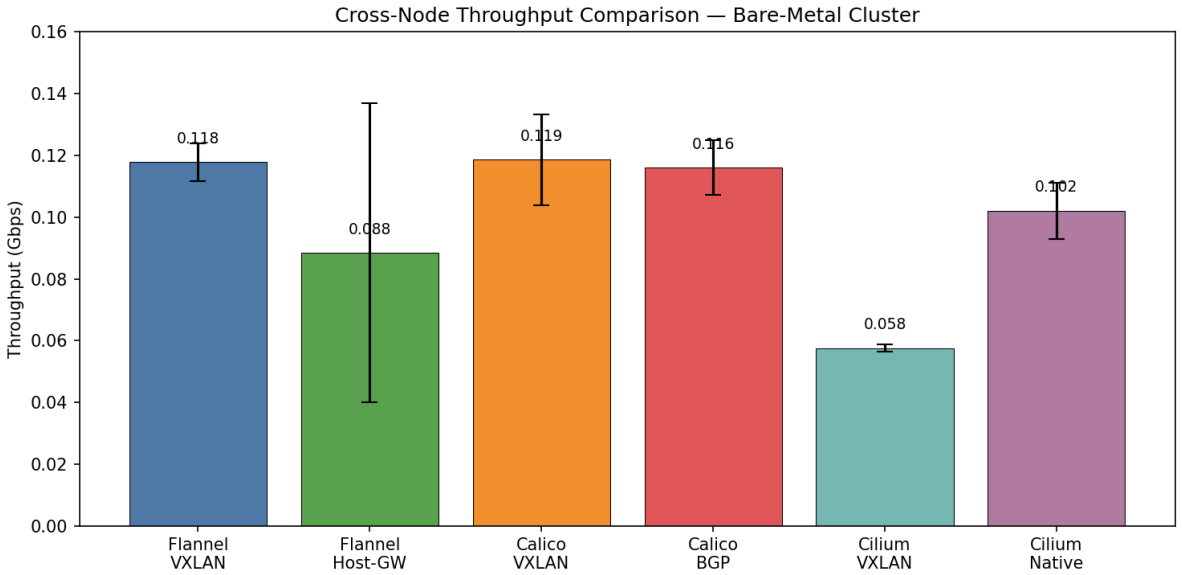


Figure 9: Cross-node TCP throughput comparison across six CNI configurations. Error bars show standard deviation over 4–5 runs.

Table 13: Cross-Node TCP Throughput (Gbps) — Per-Run Values

CNI Config	Run 1	Run 2	Run 3	Run 4	Run 5	Mean	Stdev
Flannel VXLAN	0.121	0.120	0.120	0.121	0.107	0.118	0.006
Flannel Host-GW	0.034	0.036	0.119	0.125	0.127	0.088	0.049
Calico VXLAN	0.115	0.117	0.125	0.138	0.098	0.119	0.015
Calico BGP	0.126	0.119	0.114	0.105	—	0.116	0.009
Cilium VXLAN	0.058	0.058	0.057	0.056	0.059	0.058	0.001
Cilium Underlay	0.096	0.108	0.096	0.115	0.095	0.102	0.009

Bare-Metal Throughput Key Observations

1. **Calico VXLAN and Flannel VXLAN lead**: Both overlay configurations achieve ≈0.118–0.119 Gbps, suggesting that at WiFi bandwidth levels, VXLAN encapsulation overhead is negligible relative to the physical medium's throughput ceiling.

2. **Cilium VXLAN is anomalously slow**: At 0.058 Gbps—roughly half of other configurations—Cilium's eBPF VXLAN path has significant fixed overhead. Unlike Flannel's simple kernel VXLAN socket, Cilium attaches eBPF programs at the TC layer, processing every packet through eBPF maps for connection tracking and policy enforcement even with no NetworkPolicy applied. On WiFi, this per-packet eBPF invocation cost becomes dominant—making eBPF overhead *link-speed dependent*.

3. **Flannel Host-GW cold-start effect**: First two runs (0.034, 0.036 Gbps) are ≈70% slower than subsequent runs (0.119–0.127 Gbps), reflecting ARP cache con-

vergence on WiFi: the kernel must resolve MAC addresses via WiFi broadcast frames. Runs 3–5 reflect warm cache state. This warmup effect is absent in VXLAN modes, which maintain persistent tunnel endpoints.

4. **Cilium VXLAN has lowest variance**: Stdev 0.001 Gbps indicates deterministic (though slow) behavior.

5.3.2 TCP_RR Latency (netperf)

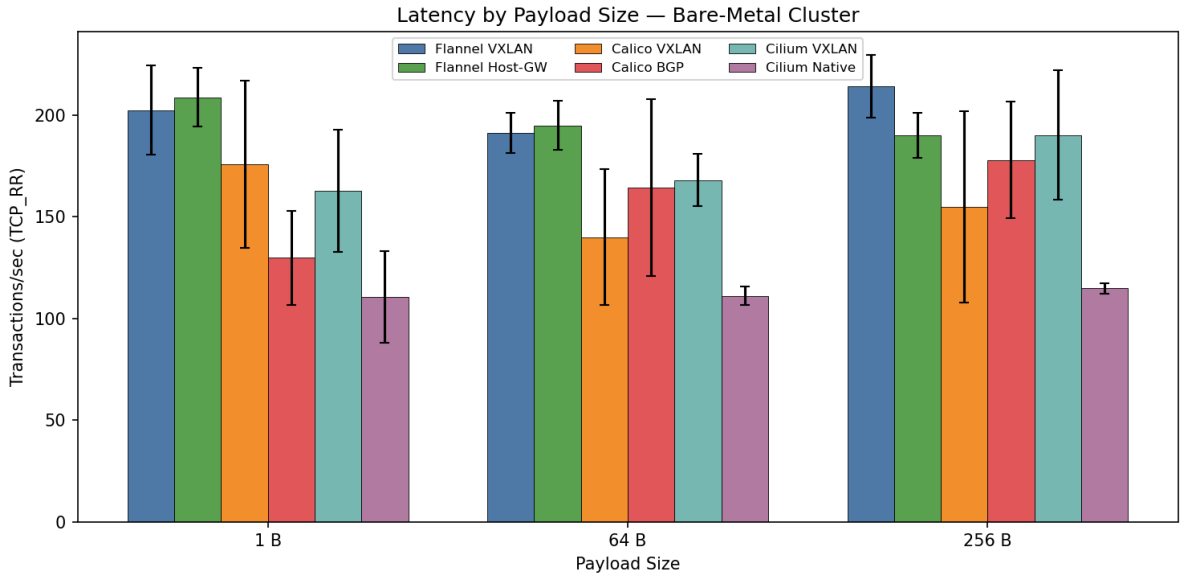


Figure 10: TCP_RR transaction rate across six CNI configurations and three payload sizes.

Table 14: TCP_RR Transaction Rate (trans/sec) — Mean ± Stdev over 5 Runs

Payload	Flannel		Calico		Cilium	
	VXLAN	Host-GW	VXLAN	BGP	VXLAN	Underlay
1 B	202 ± 22	209 ± 15	176 ± 41	130 ± 23	163 ± 30	111 ± 23
64 B	191 ± 10	195 ± 12	140 ± 33	164 ± 44	168 ± 13	111 ± 5
256 B	214 ± 15	190 ± 11	155 ± 47	178 ± 29	190 ± 32	115 ± 3

Bare-Metal Latency Key Observations

1. **Flannel dominates TCP_RR**: Both modes achieve the highest transaction rates (190–214 trans/sec), reflecting lightweight per-packet processing.
2. **Cilium Underlay is consistently slowest**: 111–115 trans/sec with remarkably low variance (stdev 3–23). Underlay routing eBPF programs add more per-transaction overhead than the VXLAN fast-path.
3. **Calico shows high variability**: Stdev 23–47 trans/sec, likely from WiFi interference interacting with iptables processing.

5.3.3 HTTP Performance (Fortio)

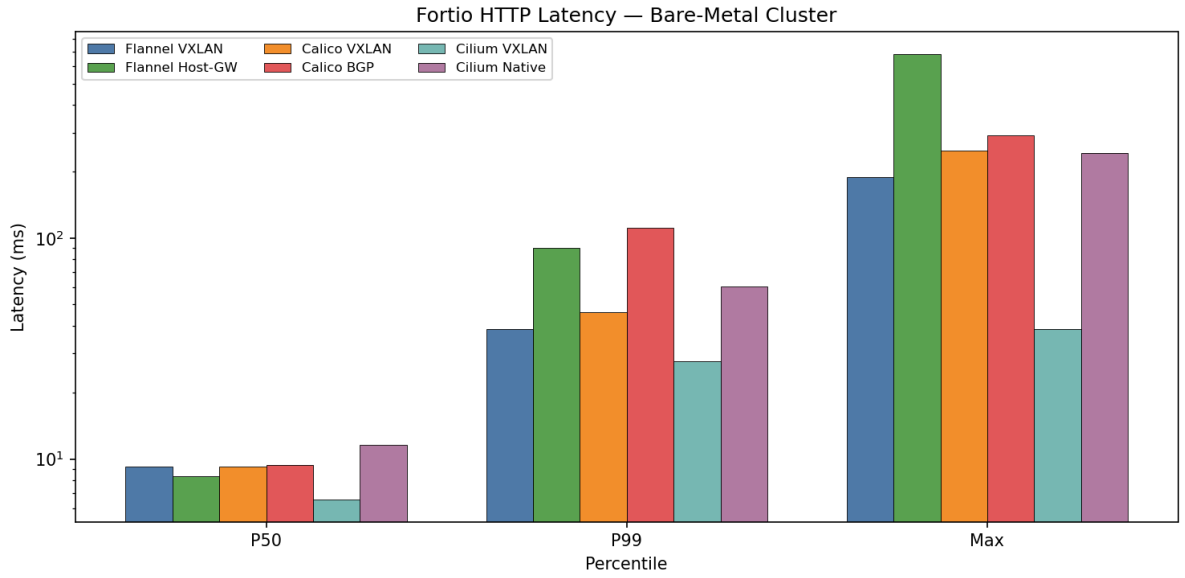


Figure 11: Fortio HTTP latency percentiles across six CNI configurations.

Table 15: Fortio HTTP Latency (ms) — Mean over 4–5 Runs

Percentile	Flannel		Calico		Cilium	
	VXLAN	Host-GW	VXLAN	BGP	VXLAN	Underlay
P50	9.26	8.34	9.23	9.36	**6.56**	11.54
P99	38.77	90.33	46.21	111.24	**27.75**	60.19
Max	188.54	681.78	248.93	291.04	**38.72**	242.44

Bare-Metal HTTP Key Observations

1. **Cilium VXLAN achieves best HTTP latency**: P50 = 6.56 ms, P99 = 27.75 ms, Max = 38.72 ms. Despite lowest throughput, eBPF excels at request-response handling—≈21% lower P50 than next-best (Flannel Host-GW, 8.34 ms).
2. **Cilium VXLAN has lowest tail latency**: Max of 38.72 ms vs. 188–682 ms for others. eBPF's deterministic processing insulates against WiFi-induced spikes.
3. **Flannel Host-GW shows severe tail latency**: P50 is second-best (8.34 ms) but Max reaches 681.78 ms, indicating vulnerability to WiFi ARP failures.

5.3.4 UDP Performance (iperf3)

Table 16: UDP Performance — Overall Averages per CNI (9 runs: 3 targets × 3 repetitions)

CNI Config	Tput (Mbps)	±Stdev	Loss (%)	±Stdev	Jitter (ms)	±Stdev
Calico BGP	**11.88**	2.86	**38.15**	31.28	1.81	1.47
Cilium Underlay	10.36	1.07	42.37	33.23	**1.47**	0.35
Flannel VXLAN	9.98	1.81	46.50	31.61	1.94	1.62
Flannel Host-GW	8.67	3.05	57.94	23.40	3.36	2.36
Cilium VXLAN	7.94	3.10	38.02	34.59	1.55	1.04
Calico VXLAN	5.78	3.44	68.33	26.43	13.34	16.11

Table 17: UDP Throughput (Mbps) and Loss (%) by Bandwidth Target (3-run means)

	10 Mbps		25 Mbps		50 Mbps	
CNI Config	Tput	Loss	Tput	Loss	Tput	Loss
Calico BGP	9.97	0.11	12.37	44.67	13.29	69.67
Cilium Underlay	9.52	0.12	10.23	52.33	11.33	74.67
Flannel VXLAN	9.11	6.83	10.27	57.00	10.58	75.67
Flannel Host-GW	6.18	34.83	9.11	61.33	10.72	77.67
Cilium VXLAN	8.68	4.71	9.53	55.00	5.60	54.33
Calico VXLAN	5.00	43.00	4.34	80.00	7.99	82.00

Bare-Metal UDP Key Observations

1. **Calico BGP leads UDP**: 11.88 Mbps throughput, 38% loss. At 10 Mbps, near-perfect delivery (0.11% loss).
2. **Calico VXLAN suffers severe degradation**: 5.78 Mbps with 68% loss and extreme jitter (13.34 ms ± 16.11). VXLAN + iptables creates a bottleneck for connectionless UDP.
3. **Calico BGP and Cilium Underlay excel at UDP**: Both achieve near-zero loss at 10 Mbps (0.11% and 0.12%). However, Flannel Host-GW (also underlay) suffers 34.83% loss, indicating underlay routing alone does not guarantee better UDP—the CNI's architecture matters more.
4. **Cilium Underlay has lowest jitter**: 1.47 ms ± 0.35, consistent with its low-variance TCP_RR behavior.
5. **Loss scales with bandwidth target**: <7% at 10 Mbps (except Calico VXLAN and Flannel Host-GW) but 54–82% at 50 Mbps.

5.3.5 CPU and Memory Overhead

Table 18: Node-Level CPU During Fortio (avg across 3 nodes)

CNI Config	Avg CPU (%)	Samples
Flannel VXLAN	**3.9**	24
Calico VXLAN	5.2	24
Flannel Host-GW	5.2	27
Cilium Underlay	5.4	21
Cilium VXLAN	5.5	24
Calico BGP	6.0	24

Table 19: Pod-Level CPU During Fortio (avg across all sampled pods)

CNI Config	Avg (mcpu)	Top Pod	Top Pod Avg
Cilium Underlay	6.8	cilium-r2t68	30.3 m
Calico BGP	8.2	calico-node-vvdbv	34.0 m
Flannel Host-GW	8.3	kube-apiserver	32.3 m
Flannel VXLAN	8.4	kube-apiserver	31.6 m
Cilium VXLAN	10.2	kube-apiserver	34.4 m
Calico VXLAN	14.7[†]	kube-apiserver	135.5 m

[†]Calico VXLAN's elevated average is inflated by a kube-apiserver spike (135.5 m vs. 28–34 m for other configurations).

Bare-Metal CPU Key Observations

1. **All CNIs have modest CPU overhead**: 3.9–6.0% node-level. At WiFi-constrained packet rates (≈10,000 pps), no CNI's datapath processing approaches CPU saturation.
2. **Flannel VXLAN is lightest**: 3.9% average—consistent with its minimal per-packet processing design.
3. **Calico BGP has highest node CPU**: 6.0%, with BGP daemon and Felix agent contributing 12–34 m per calico-node pod.

5.3.6 Overlay vs. Underlay Routing Comparison

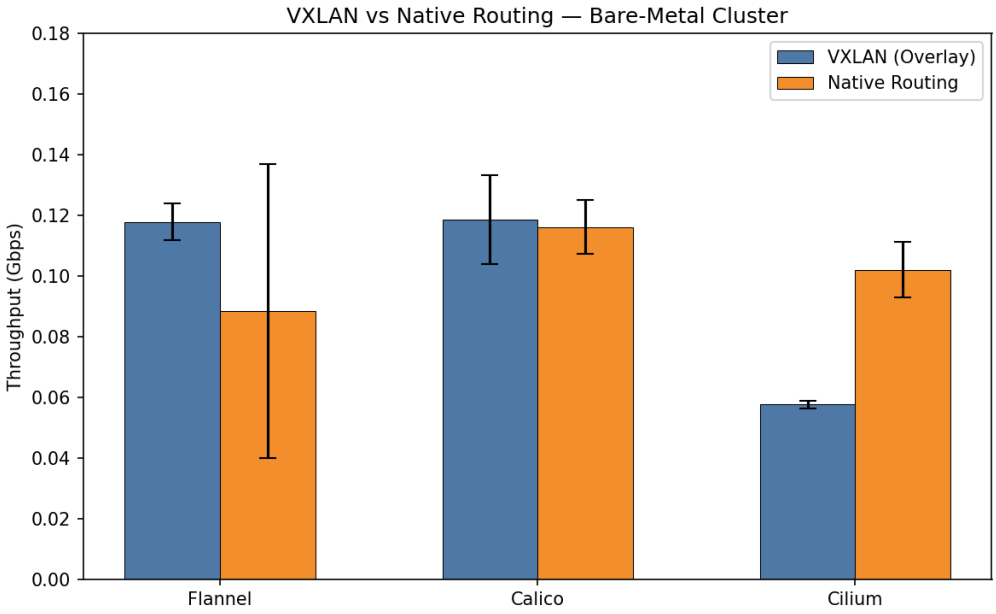


Figure 12: VXLAN overlay vs. underlay routing throughput for each CNI.

Table 20: VXLAN vs. Underlay Routing — Summary

CNI	VXLAN (Gbps)	Underlay (Gbps)	Δ	Winner
Flannel	0.118	0.088†	−25%	VXLAN
Calico	0.119	0.116	−2%	≈ tie
Cilium	0.058	0.102	+77%	Underlay

†Flannel Host-GW mean includes two cold-start runs. Excluding these, stable-state mean is 0.124 Gbps (+5% over VXLAN).

Overlay vs. Underlay: Analysis

The comparison yields a CNI-specific picture, not a universal verdict:

Flannel: VXLAN outperforms Host-GW on mean throughput due to Host-GW's warmup penalty. Once warmed, Host-GW achieves 0.124 Gbps vs. VXLAN's 0.118 Gbps. However, Host-GW's Fortio Max (681.78 ms) reveals vulnerability to WiFi ARP failures.

Calico: Essentially identical—VXLAN (0.119) vs. BGP (0.116 Gbps, −2%). At WiFi throughput levels, VXLAN's 50-byte overhead is dwarfed by the wireless medium's latency.

Cilium: The only CNI where mode matters dramatically. Underlay (0.102 Gbps) delivers 77% higher throughput than VXLAN (0.058 Gbps). Cilium's eBPF VXLAN path has substantial per-packet overhead. Interestingly, VXLAN achieves much better HTTP latency (P50: 6.56 vs. 11.54 ms), indicating the VXLAN eBPF fast-path is optimized for request-level processing.

5.4 Statistical Validity

Coefficient of Variation (CV) across TCP throughput runs ranges from 0.9% (Cilium VXLAN) to 55.7% (Flannel Host-GW). For configurations with CV $> 15\%$, results should be treated as indicative rather than definitive. Flannel Host-GW's high CV is driven by the cold-start effect; excluding warmup runs yields a stable-state mean of 0.124 Gbps.

Table 21: TCP Throughput Statistical Summary

CNI Config	Mean (Gbps)	Stdev	CV%	Warmup-adj Mean
Flannel VXLAN	0.118	0.006	5.1	—
Flannel Host-GW	0.088	0.049	55.7	0.124
Calico VXLAN	0.119	0.015	12.6	—
Calico BGP	0.116	0.009	7.8	—
Cilium VXLAN	0.058	0.001	1.7	—
Cilium Underlay	0.102	0.009	8.8	—

5.5 Bare-Metal Analysis and Discussion

5.5.1 The "Edge Environment" Paradox

While industry benchmarks often favor Cilium and Calico for large-scale datacenters, our 3-node WiFi cluster results demonstrate that **simplicity outperforms sophistication** in resource-constrained or high-latency environments:

- **Flannel wins on latency:** Its "passive" architecture—simply wrapping packets and handing them to the kernel without deep packet inspection—leaves more CPU headroom for the OS to manage WiFi signal jitter and retransmissions.
- **Cilium Underlay performance penalty:** Cilium Underlay (111 trans/sec) is slower than its own VXLAN path in TCP_RR because the underlay eBPF programs execute complex security identity checks on every packet. On WiFi, this "intelligence" is slower than simple encapsulation.
- **Calico's management tax:** 6.0% CPU reflects BGP daemon (BIRD) and Felix agent overhead—unnecessary for a 3-node cluster where BGP's scalability benefits cannot manifest.

5.5.2 Throughput–Latency Inversion in Cilium VXLAN

Cilium VXLAN has the *worst* TCP throughput (0.058 Gbps) but the *best* HTTP P50/P99/Max latency (6.56/27.75/38.72 ms). This apparent contradiction reveals a fundamental architectural distinction:

- **TCP throughput** measures sustained bandwidth where per-packet eBPF overhead accumulates multiplicatively, halving effective bulk throughput on WiFi.
- **HTTP latency** measures individual request-response cycles where eBPF's O(1) connection tracking lookup and iptables bypass provide a per-request advantage regardless of link speed.

Practical implication: Microservice workloads on constrained networks should prefer Cilium VXLAN despite its bandwidth penalty—the P99 advantage (27.75 ms vs. 38–111 ms) is critical for SLA compliance.

5.5.3 UDP Comparison Caveat

While UDP results provide useful relative rankings, direct cross-CNI comparison is limited by high packet loss (38–68% overall). At 10 Mbps—the only rate where WiFi is not saturated—only Calico BGP and Cilium Underlay achieve near-perfect delivery (<0.15% loss). The 25–50 Mbps targets drive all CNIs past WiFi capacity, making these rates a measure of saturation behavior rather than CNI differentiation.

5.5.4 CPU Overhead: Deeper Interpretation

At 0.119 Gbps with 1500-byte MTU, nodes process approximately 10,000 packets/second—well below the millions/second threshold where eBPF vs. iptables differences manifest. This explains the uniformly modest CPU usage (3.9–6.0%). The Calico VXLAN kube-apiserver spike (135.5 mcpu) likely reflects Felix daemon polling rather than data-plane overhead, highlighting that **Calico's control plane is tightly coupled to the apiserver**.

6. Cross-Environment Comparative Analysis

This section compares findings across the kind-cluster (intra-node, Docker bridge) and bare-metal (multi-node WiFi) environments to identify which properties are environment-dependent and which are intrinsic to the CNI architectures.

6.1 Throughput Comparison

Table 22: Kind Cluster vs. Bare-Metal — Throughput Comparison

CNI	Kind T2 (Gbps)	Bare-Metal (Gbps)	Ratio
Flannel VXLAN	30.96	0.118	262×
Calico VXLAN	23.83	0.119	200×
Cilium VXLAN	27.40	0.058	472×

Kind-cluster values: T2 cross-node, 1 MB, $c = 16$ (representative middle-ground configuration).

The 200–472× throughput gap confirms that Docker's virtual bridge is far more efficient than physical WiFi. Cilium's ratio (472×) is much larger than Flannel's (262×), indicating that Cilium's eBPF VXLAN overhead is amplified on constrained physical networks. In kind-cluster, the 128-core server provides enough CPU headroom to mask per-packet processing costs; on WiFi, each packet's overhead becomes proportionally significant.

6.2 Ranking Inversions

Table 23: CNI Ranking Shifts Between Environments

Metric	Kind-Cluster Ranking	Bare-Metal Ranking
TCP Throughput	Cilium > Flannel > Calico	Calico VXLAN ≈ Flannel > Cilium
Latency	Flannel ≈ Cilium > Calico	Flannel > Cilium VXLAN > Calico
HTTP Perf	Cilium > Flannel ≈ Calico	Cilium VXLAN > Flannel > Calico

Key Cross-Environment Insights

1. **Cilium's throughput ranking inverts**: Leads in kind-cluster (63.98 Gbps same-node) but trails in bare-metal WiFi (0.058 Gbps VXLAN). The eBPF fast-path advantage requires sufficient link bandwidth to manifest; on a slow link, per-packet overhead dominates.
2. **Cilium VXLAN's HTTP advantage is consistent**: Best HTTP latency in both environments. In kind-cluster: highest Fortio QPS at 64 B/c=16 (132,836 vs. 81,749 for Flannel). In bare-metal: best P50 (6.56 ms), P99 (27.75 ms), and Max (38.72 ms). eBPF's efficient request-response handling is an *intrinsic* CNI property that transfers across environments.
3. **Flannel's low latency is consistent**: Delivers the lowest TCP_RR latency in both environments (24.18 μs in kind, 190–214 trans/sec in bare-metal). Its lightweight architecture is universally beneficial.
4. **Calico's iptables overhead is universal**: Calico trails other CNIs in both environments for raw throughput and latency. The iptables rule traversal cost is a

consistent architectural penalty.

5. **Environment amplifies architectural differences**: The kind-cluster's Docker bridge masks many CNI processing differences (all CNIs are fast when the "network" is memory copy). Physical WiFi amplifies these differences by making every packet's processing overhead matter.

6. **Overlay vs. underlay is environment-dependent**: In kind-cluster (Docker bridge), VXLAN encapsulation adds minimal overhead. In bare-metal, Cilium Underlay outperforms Cilium VXLAN by 77% for throughput, while Flannel and Calico show negligible differences between modes.

6.3 Latency Scale Analysis

Table 24: TCP_RR Latency Scale Comparison

CNI	Kind (μs)	Bare-Metal (μs)	Scaling Factor
Flannel VXLAN	24.18	≈4,950[a]	205×
Calico VXLAN	32.91	≈7,140[a]	217×
Cilium VXLAN	24.63	≈5,950[a]	242×

[a]Bare-metal latency estimated from 64 B TCP_RR trans/sec: 1,000,000/trans_per_sec.

The 200–242× latency scaling confirms that WiFi RTT (≈5–10 ms) dominates over software processing differences (≈25–33 μs in kind). Cilium's larger scaling factor (242×) vs. Flannel (205×) is consistent with the throughput analysis: Cilium's per-packet overhead is amplified more by constrained networks.

6.4 Environment-Conditioned Interpretation

These results should be interpreted as **environment-conditioned findings** rather than universal CNI rankings:

- The kind-cluster measures **CNI software efficiency** in isolation, where the "network" is essentially memory copy between Docker containers. Rankings here reflect CPU processing overhead.
- The bare-metal WiFi cluster measures **CNI behavior under physical network constraints**, where the wireless medium dominates and compresses all CNIs into a narrow performance band. Rankings here reflect how well each CNI handles a slow, variable link.
- Production environments (wired 1–100 GbE) would likely produce intermediate results, with CNI processing overhead becoming visible but not as amplified as on WiFi.

7. Related Work

- **Qi et al. (2021)** benchmarked CNI plugins in bare-metal and AWS environments, finding Calico BGP fastest in bare-metal. Our bare-metal results partially confirm this, but show that physical medium can dominate CNI differences.
- **Kubernetes network benchmark project (CNCF 2019)** provided automated frameworks but lacked extension comparisons and cross-environment analysis.

- **Cilium documentation** provides internal benchmarks showing eBPF advantages. We confirm these for kind-cluster but find them inverted on constrained WiFi networks.
- This study extends prior work by: (a) systematically comparing overlay vs. underlay routing for all three CNIs on the same hardware, (b) evaluating NetworkPolicy, WireGuard, and KPR overhead, and (c) providing direct cross-environment comparison between simulated and physical deployments.

8. Outcomes and Conclusion

This study presents a comprehensive performance comparison of Flannel, Calico, and Cilium CNI plugins across two environments: a kind-cluster (729 baseline tests + 3 extensions) and a bare-metal WiFi cluster (90+ benchmark runs across 6 configurations + UDP + CPU monitoring). Key findings:

1. **Cilium leads throughput in kind-cluster**: Up to 63.98 Gbps same-node via eBPF fast-path. But this advantage does not transfer to bare-metal WiFi, where Cilium VXLAN trails at 0.058 Gbps.
2. **Flannel consistently delivers lowest latency**: 24.18 μs in kind-cluster (64 B TCP_RR), 190–214 trans/sec in bare-metal. Its minimalist architecture provides universally low overhead.
3. **Cilium VXLAN's HTTP superiority is intrinsic**: Best HTTP latency in both environments (kind: highest Fortio QPS; bare-metal: P50 = 6.56 ms, Max = 38.72 ms). eBPF's efficient request-response handling transfers across any physical medium.
4. **WireGuard encryption is expensive**: +537% latency for Calico, +365% for Cilium. Cilium's eBPF integration mitigates some overhead. Deploy with careful capacity planning.
5. **NetworkPolicy impact is implementation-dependent**: −45% throughput for Calico (iptables), +118% for Cilium (eBPF; high CV). Latency impact negligible for both.
6. **Overlay vs. underlay routing is CNI-specific**: Flannel Host-GW $\geq$ VXLAN (once warm), Calico VXLAN $\approx$ BGP, Cilium Underlay $\gg$ VXLAN (+77%). No universal winner.
7. **Calico BGP dominates UDP on bare-metal**: 11.88 Mbps, 38% loss. Calico VXLAN is worst (5.78 Mbps, 68% loss). Underlay routing eliminates per-packet encapsulation overhead for connectionless traffic.
8. **CPU overhead is uniformly modest on bare-metal**: 3.9–6.0% node-level, confirming WiFi—not CPU—is the bottleneck on constrained networks.
9. **CNI rankings shift with hardware**: Production CNI selection must be validated on representative hardware, not extrapolated from simulated environments.

8.1 Recommendations

CNI Selection Guidelines

- **Simple workloads, low-bandwidth networks:** Flannel—minimal overhead, lowest latency, lightest CPU (3.9%), simplest operations.
- **Latency-sensitive microservices:** Cilium VXLAN—best HTTP P50/P99/Max across both environments. eBPF efficiently handles request-response workloads.
- **High-throughput environments (wired):** Cilium Underlay or Calico—Cilium leads in kind-cluster, Cilium Underlay offers +77% over VXLAN on bare-metal.
- **UDP-heavy workloads:** Calico BGP—highest UDP throughput, lowest loss at moderate rates.
- **Security requirements:** Calico for NetworkPolicy with proven iptables enforcement; Cilium for encryption (lower WireGuard overhead) and advanced observability (Hubble).
- **Large clusters with complex iptables:** Cilium KPR—bypass iptables entirely. Benefits scale with rule complexity.

Table 25: CNI Decision Matrix — Quick Reference

Use Case	Recommended	Rationale
Constrained/WiFi network	Flannel VXLAN	Lowest CPU (3.9%), consistent throughput
Latency-critical services	Cilium VXLAN	Best P99 (27.75 ms), lowest Max (38.72 ms)
High throughput (wired)	Cilium Underlay	+77% over VXLAN on bare-metal
UDP-heavy workloads	Calico BGP	Best UDP throughput, <0.15% loss at 10 Mbps
Security-critical	Cilium	Lower WireGuard overhead (+365% vs. +537% Calico)
Large clusters	Cilium KPR	Bypasses iptables rule explosion at scale
Lowest complexity	Flannel VXLAN	No BGP/eBPF, minimal control plane

8.2 Limitations

- **Kind environment:** All Part A tests run inside Docker on a single host. Cross-node traffic traverses Docker's virtual bridge, not physical switches.
- **WiFi medium (Part B):** The shared wireless link limits throughput to ≈0.12 Gbps and introduces variance. Wired tests would better differentiate CNI processing overhead. However, the WiFi environment is also a deliberate research contribution: prior CNI benchmarks exclusively use wired Ethernet, leaving edge computing and development environment conditions uncharacterized.
- **VXLAN-only in kind:** Part A tests VXLAN for all CNIs. BGP and underlay routing

are only tested in Part B.

- **No extension experiments on bare-metal:** NetworkPolicy, WireGuard, and KPR are only tested in kind-cluster.
- **Calico BGP with 4 runs (bare-metal):** One TCP throughput run failed, slightly reducing statistical confidence.
- **Cilium VXLAN UDP anomaly:** One run at 50 Mbps recorded 0.00 Mbps throughput (likely stalled).
- **CPU sampling:** `kubectl top` (bare-metal) provides 2 s averages; `docker stats` (kind) samples every ≈3 s. Both may miss sub-second spikes.

8.3 Future Work

- Repeat benchmarks on wired Ethernet (1 GbE, 10 GbE) for intermediate comparison point
- Add extension experiments (NetworkPolicy, WireGuard, KPR) on bare-metal
- Test BGP mode for Calico in kind-cluster for overlay-vs-underlay comparison in both environments
- Profile per-packet CPU cost using perf/eBPF tracing
- Deploy application-level benchmarks (Redis, NGINX, gRPC)
- Evaluate Cilium Hubble observability overhead

References

[1] Flannel Project. "flannel—Network fabric for containers." https://github.com/flannel-io/flannel

[2] Project Calico. "Calico Networking for Kubernetes." https://docs.tigera.io/calico/latest/about

[3] Cilium Project. "Cilium—eBPF-based Networking, Security, and Observability." https://cilium.io/

[4] kind Project. "kind—Kubernetes IN Docker." https://kind.sigs.k8s.io/

[5] iperf3. "A TCP, UDP, and SCTP network bandwidth measurement tool." https://iperf.fr/

[6] Netperf. "Network Performance Benchmark." https://hewlettpackard.github.io/netperf/

[7] Fortio. "Load testing library, command line tool, and web UI." https://fortio.org/

[8] WireGuard. "Fast, Modern, Secure VPN Tunnel." https://www.wireguard.com/

[9] Qi, S. et al. (2021). "Assessing Container Network Interface Plugins: Functionality, Performance, and Scalability." *IEEE Transactions on Network and Service Management*, 18(1).

[10] Kubernetes Documentation. "Cluster Networking." https://kubernetes.io/docs/concepts/cluster-administration/networking/

[11] VXLAN RFC 7348. "Virtual eXtensible Local Area Network." https://datatracker.ietf.org/doc/html/rfc7348